Java TreeSet – Complete Notes

Simple Roman Hindi | Basics se LeetCode application tak

1. TreeSet kya hota hai?

Java ka **TreeSet** ek collection hai jo elements ko automatically sorted order (default ascending order) mein rakhta hai. Ismein duplicate elements store nahi hote. TreeSet ko tab use karte hain jab sorted data ke saath fast searching ki zaroorat ho.

Simple example: Tum 30, 10, 50, 20 aur phir 10 add karte ho. TreeSet output ko [10, 20, 30, 50] rakhega. 10 duplicate tha, isliye ek hi baar dikhega.

Code example

```
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {
        TreeSet set = new TreeSet<>();

        set.add(30);
        set.add(10);
        set.add(50);
        set.add(20);
        set.add(10);

        System.out.println(set);
    }
}

// Output: [10, 20, 30, 50]
```

2. TreeSet ki main properties

Property	Meaning
Sorted order	Elements automatically ascending order mein arrange hote hain.
No duplicates	Same value ko baar-baar add karne par bhi ek hi copy rehti hai.
Null	Natural ordering ke saath null generally allow nahi hota.
Search / insert	add, remove aur search operations generally O(log n) hote hain.

3. Important TreeSet methods

Method	Kaam	Example
add(x)	Element insert karta hai	set.add(8)
remove(x)	Element remove karta hai	set.remove(8)
contains(x)	Check karta hai element present hai ya nahi	set.contains(8)
first()	Sabse chhota element deta hai	set.first()
last()	Sabse bada element deta hai	set.last()
ceiling(x)	x se bada ya equal sabse chhota element	set.ceiling(7)

floor(x)	x se chhota ya equal sabse bada element	set.floor(7)
higher(x)	x se strictly bada sabse chhota element	set.higher(8)
lower(x)	x se strictly chhota sabse bada element	set.lower(8)

4. ceiling() ko detail mein samjho

ceiling(x) TreeSet mein **x ke equal ya x se bade values** mein se sabse chhoti value return karta hai. Agar aisi koi value nahi milti, to null return hota hai.

Maan lo TreeSet mein [2, 5, 8, 12, 15] hain:

Call	Return	Reason
ceiling(7)	8	7 se bada ya equal sabse chhota number 8 hai.
ceiling(8)	8	Equal value allowed hai.
ceiling(9)	12	9 se bada sabse chhota number 12 hai.
ceiling(16)	null	16 ya usse bada koi number nahi hai.

5. TreeSet aur Prefix Sum ka connection

Max Sum of Rectangle No Larger Than K problem mein pehle 2D matrix ko ek 1D array mein convert karte hain. Phir us 1D array ke subarrays ka sum find karna hota hai. Is step mein Prefix Sum aur TreeSet saath use hote hain.

Subarray sum ka formula

Subarray Sum = Current Prefix Sum - Previous Prefix Sum

Humein chahiye ki subarray sum k se bada na ho:

```
currentPrefix - previousPrefix <= k
```

Is inequality ko rearrange karne par:

```
previousPrefix >= currentPrefix - k
```

Ab humein TreeSet mein aisa previous prefix chahiye jo currentPrefix - k se bada ya equal ho, aur inmein sabse chhota ho. Bilkul yahi kaam **ceiling()** karta hai.

6. Numerical dry run

Array = [2, -1, 3], k = 3

Step	Current element	Current prefix	TreeSet se pehle	ceiling(prefix-k)	Subarray sum
Start	—	0	{0}	—	—
1	2	2	{0}	ceiling(-1) = 0	2 - 0 = 2
2	-1	1	{0, 2}	ceiling(-2) = 0	1 - 0 = 1
3	3	4	{0, 1, 2}	ceiling(1) = 1	4 - 1 = 3

Har step ke baad current prefix ko TreeSet mein add karte hain. Is dry run mein valid subarray sums 2, 1 aur 3 mile; maximum valid sum 3 hai, jo k ke equal hai.

7. LeetCode code mein important lines

```
TreeSet set = new TreeSet<>();
set.add(0);

int prefixSum = 0;

for (int num : sums) {
    prefixSum += num;

    Integer previous = set.ceiling(prefixSum - k);

    if (previous != null) {
        answer = Math.max(answer, prefixSum - previous);
    }

    set.add(prefixSum);
}
```

Line-by-line meaning:

1. TreeSet banao: previous prefix sums sorted form mein store honge.
2. set.add(0): empty prefix ko store karo, taaki array ke index 0 se start hone wale subarrays bhi consider hon.
3. prefixSum += num: current position tak ka cumulative sum update karo.
4. ceiling(prefixSum - k): required condition satisfy karne wala smallest previous prefix dhoondo.
5. prefixSum - previous: valid subarray ka sum.
6. Math.max: ab tak ka maximum valid sum update karo.
7. set.add(prefixSum): current prefix ko future subarrays ke liye save karo.

8. Complexity

Operation	Time
TreeSet add(x)	O(log n)
TreeSet ceiling(x)	O(log n)
TreeSet contains(x)	O(log n)
TreeSet remove(x)	O(log n)
TreeSet first() / last()	O(log n) worst-case; implementations may provide these efficiently.

9. Quick revision

- TreeSet = sorted + unique elements.
- ceiling(x) = x ya x se bada sabse chhota element.
- floor(x) = x ya x se chhota sabse bada element.
- Prefix Sum subarray sum ko currentPrefix - previousPrefix se nikalta hai.
- Max subarray sum <= k ke liye previousPrefix >= currentPrefix - k chahiye.
- Isliye TreeSet.ceiling(currentPrefix - k) use hota hai.

Yaad rakho: TreeSet khud Prefix Sum nahi banata. Hum Prefix Sum calculate karte hain aur TreeSet mein previous Prefix Sum values store karte hain, taaki required value ko efficiently search kar sakein.